Neotech Faculty of Diploma Engineering

LAB MANUAL OF

"FUNDAMENTALS OPERATING SYSTEM"

(DI03000041)

3rd semester

Department of

Computer Engineering

A.Y. 2025-26

CERTIFICATE

This is to certify that Mr. / Ms.....with enrollment no……Has successfully completed his / her laboratory experiments in the subject (DI03000041)…….…….. .. From the department of…….. During the Academic year…….

Date: …….. *Faculty Signature: ……*

COMPUTER ENGINEERING
DIPLOMA 3rd SEM
SUBJECT: FOS
SUBJECT CODE: DI03000041

List of Practical

Sr. No	Name of the Experiment	Page No	Date of Submission	Sign
1.	Compare Windows, Linux OS and Mac OS. (latest version)			
2.	Process Scheduling algorithm a) UsingFCFS, SJFandRoundrobinalgorithm, drawGanttchart for Process P0, P1, P2, and P3. Arrival time 0, 1, 2, 3 and process execution Time 5, 3, 8, 6 respectively. Find Average turnaround time and average waiting time. UsingFCFS, SJFandRoundrobinalgorithm, drawGanttchart for Process P0, P1, P2, and P3. Arrival time 3, 0, 4, 5 and process execution Time 5, 4, 2, 4 respectively. Find Average turnaround time and average waiting time.			
3.	SolveexamplesusingFirstfit, Best fit, Worst fit Algorithms where Jobs J1 with 20KB , J2 with 200KB , J3 with 500KB, J4 with 50KB, J5 with 150 KB memory requests Consider memory partitions of size 50 KB, 300 KB, 75 KB, 700 KB and 100 KB.			
4.	Solve Using LRU and FIFO. Consider the page reference string of size 12: 1, 2, 3, 4, 5, 1, 3, 1, 6, 3, 2, 3 with frame size 4(i.e. maximum 4 pages in a frame).			
5.	Consider a disc with 200 tracks (0-199) and a disc queue with the following input/output requests: 75, 90, 40, 135, 50, 170, 65, 10. The Read/Write head's initial position is 46, and it will move to the left-hand side. Using the SCAN			

	and CSCAN method, determine the total number of track movements of the Read/Write head.			
6.	Test and run basic Unix commands.			
7.	Test and run advanced Unix commands.			
8.	Test commands related with File editing with Vi, Vim, gedit, gcc.			
9.	Create a shell script to read from command line and print "Hello".			
10.	Create a Shell script to read and display content of a file. And append content of one file to another			
11.	Create a Shell script to accept a string in lower case letters from a user, & convert to upper case letters.			
12.	Create a Shell script to add two numbers.			

Experiment No: 1

Aim : Compare windows and Linux OS (latest version)

Linux	Windows
Linux is a open source operating system.	While windows are the not the open source operating system
Linux is free of cost.	While it is costly.
It's file name case-sensitive	While it's file name is case-insensitive
In Linux, monolithic kernel is used	While in this, micro kernel is used
Linux is more efficient in comparison of windows.	While windows are less efficient.
Linux is widely used in hacking purpose based system	While windows dose not provide much efficiency in hacking.
Linux provide more security than windows.	While it provides less security than Linux.

Experiment No: 2

Aim : Solve below given example with SJF, FCFS and Round robin algorithm, draw Gantt chart.

❖ FCFS

Process	**Arrival Time(TO)**	**Execution Time OR burst time(ΔT)**
PO	O	5
Pl	1	3
P2	2	8
P3	3	6

❖ Gantt Chart:

P0	Pl	P2	P3	P4
0	3	8	16	22

Process	Arrival Time (TO)	Execution time OR burst time (ΔT)	Completio n (T.A.T=Tl – TO)	Turnaroun d Time (ta	Waiting (T.A .T-ΔT)
P0	0	5	5	5	0
Pl	1	3	8	7	4
P3	2	8	16	14	6
P4	3	6	22	19	13

Average Turnaround time = [Completion Time – Arrival Time]/ total no. processes

= [(5 - O) + (8 - l) + (l6-2) + (22-3)]/4

PO Pl P2 P3

P4

= [(5 + 7 + l4 + l9]/4

= 45/4

= ll.25 ms

Average Waiting time

= [Turnaround Time – burst / Total no. of processes

= [(5 – 5) + (7 – 3) + (l4 – 8) + (l9 – 6)]/4

PO Pl P2 P3

P4

= (O + 4 + 6 + l3] /4

= 23/4

=5.75 ms

❖ **SJF**

Process	Arrival Time(TO)	Execution Time OR burst time(ΔT)
PO	0	lO
Pl	l	6
P2	3	2
P3	5	4

Turnaround time = Completion – Arrival Time Waiting time = Turnaround time – Burst Time

❖ Gantt Chart:

0 lO l2 l6 22

Process	Arrival Time (TO)	Execution time OR burst time (ΔT)	Completio n (T.A.T=Tl – TO)	Turnaroun d Time (ta	Waiting (T.A .T-ΔT)
PO	0	lO	lO	lO	0
Pl	l	6	22	2l	l5
P3	2	2	l2	9	7
P4	3	4	l6	Ll	7

Average Turnaround time = [Completion time – Arrival time] / Total no processes

= [(lO-O) + (22-l) + (l2 -3) + (l6-5)] / 4

PO Pl P4 P3

= [lO + 2l + 9 + ll] / 4

= 5l/4

= l2.75.ms

Average Waiting time = [Turnaround time – Burst time] / total no processes

= [(lO - lO) + (2l – 6) + (9 – 2) + (ll – 4) / 4

PO Pl P2 P3

= [0+l5+7+7] /4

= 29/4

= 7.25.ms

❖ RR' Round Robin

Process	Execution time OR burst time (ΔT)
Pl	24
P2	3
P3	3

Turnaround time = Completion time – Arrival time

Waiting time = turnaround time – burst time

❖ Gantt chart:

Pl	P2	P3	Pl	Pl	Pl	Pl	Pl

0 4 7 lO l4 l8 22 26 30

Process	Execution time OR burst time (ΔT)	Completion (T.A.T=Tl – TO)	Turnaround Time (ta	Waiting (T.A .T-ΔT)
Pl	24	30	30	6
P2	3	7	7	4
P3	3	lO	lO	7

Average Turnaround time = (Completion time – Arrival time) /4 total no. of processes

= [(30 – 0) + (7 – 0) + (l0 – 0) / 3

= [30 + 7 + l0] /3

= l5.67.ms

Average Waiting time = (Turnaround time – burst time) / Total no. of processes

= [(30 – 24) + (7 – 3) + (l0 – 3) / 3

= [6 + 4 + 7] /3

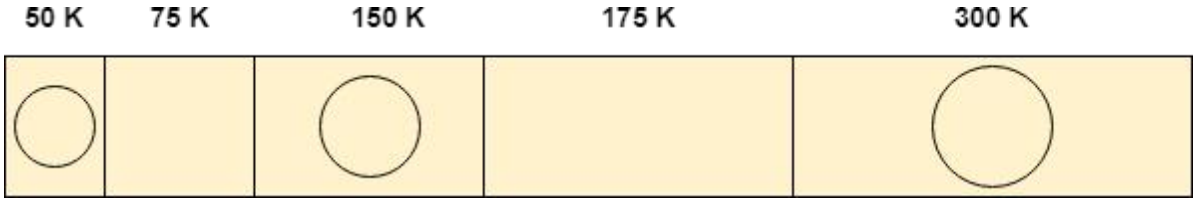
= l7/3

=5.6.ms

Experiment No: 3

Aim : Processes requests are given as:

25k, 50k, l00k, 75k

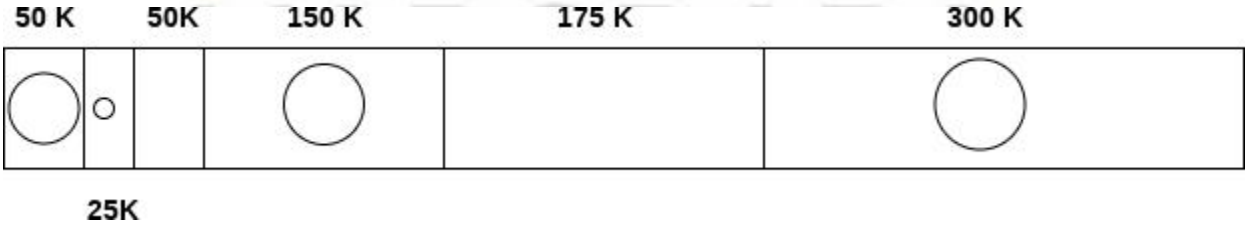


Solve above example using following algorithm:

1. First fit
2. Best fit
3. Worst fit

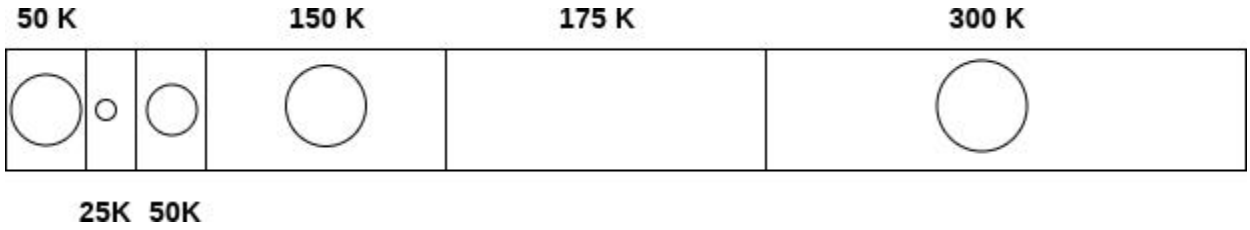
1. 25 K requirement

The algorithm scans the list until it gets first hole which should be big enough to satisfy the request of 25 K. it gets the space in the second partition which is free hence it allocates 25 K out of 75 K to the process and the remaining 50 K is produced as hole.



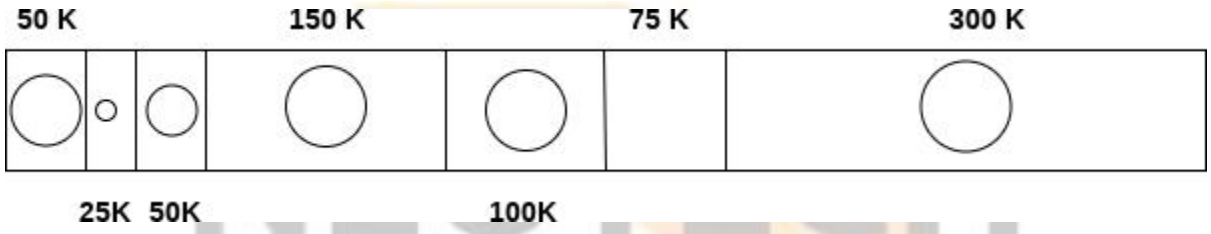
1. 50 K requirement

The 50 K requirement can be fulfilled by allocating the third partition which is 50 K in size to the process. No free space is produced as free space.



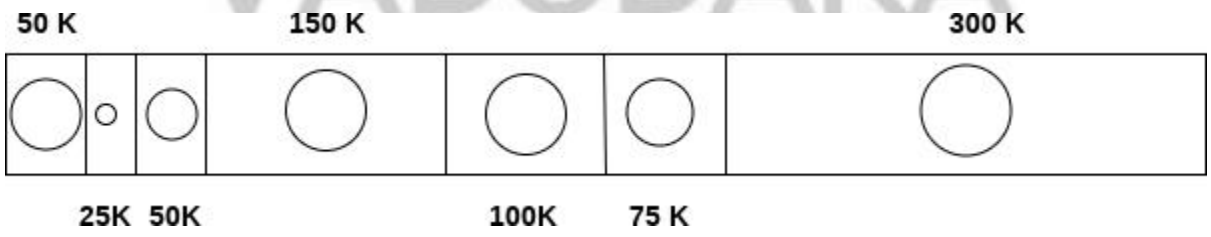
2. 100 K requirement

100 K requirement can be fulfilled by using the fifth partition of 175 K size. Out of 175 K, 100 K will be allocated and remaining 75 K will be there as a hole.



2. 75 K requirement

Since we are having a 75 K free partition hence we can allocate that much space to the process which is demanding just 75 K space.



Using first fit algorithm, we have fulfilled the entire request optimally and no useless space is remaining.

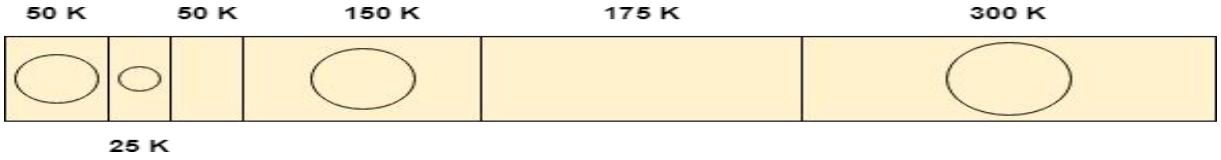
Let's see, How Best Fit algorithm performs for the problem.

Using Best Fit Algorithm

1. 25 K requirement

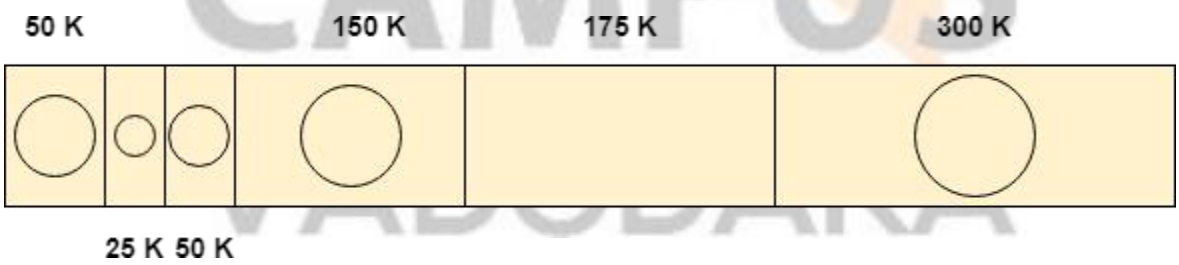
To allocate 25 K space using best fit approach, need to scan the whole list and then we find that a 75 K partition is free and the smallest among all, which can accommodate the need of the process.

Therefore 25 K out of those 75 K free partition is allocated to the process and the remaining 5o K is produced as a hole.



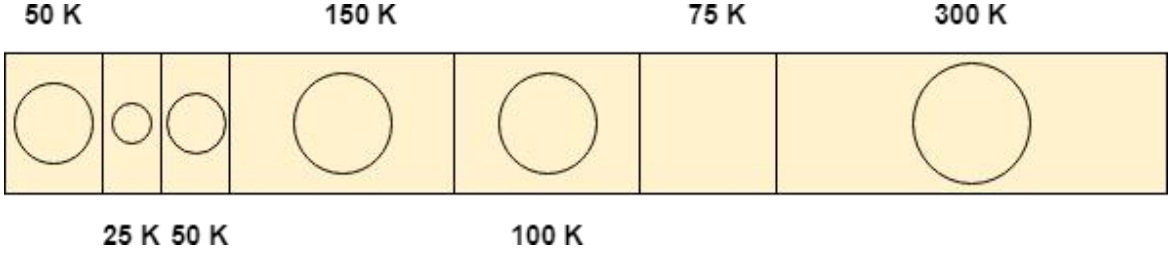
1. 50 K requirement

To satisfy this need, we will again scan the whole list and then find the 50 K space is free which the exact match of the need is. Therefore, it will be allocated for the process.



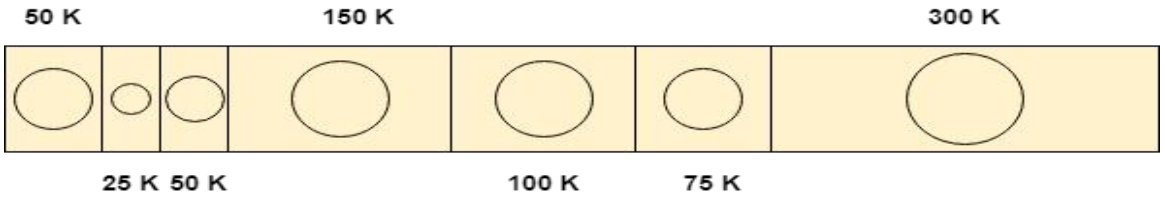
2. 100 K requirement

100 K need is close enough to the 175 K space. The algorithm scans the whole list and then allocates 100 K out of 175 K from the 5th free partition.



3. 75 K requirement

75 K requirement will get the space of 75 K from the 6th free partition but the algorithm will scan the w

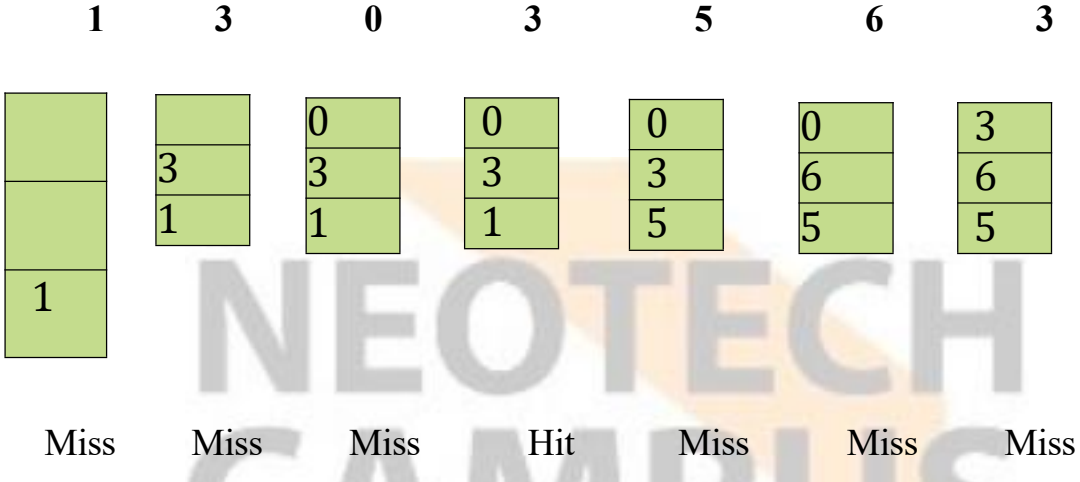


Experiment No: 4

Aim : Page replacement algorithms

i. First in First out (FIFO) - Consider page reference string 1, 3, 0, 3, 5, 6, 3 with 3 page frames. Find the number of page faults.

Page reference: 1,3,0,3,5,6,3



➔Total Page Fault = 6

- Initially, all slots are empty, so when 1, 3, 0 came they are allocated to the empty slots —> 3 Page Faults.
- when 3 comes, it is already in memory so —> 0 Page Faults. Then 5 comes, it is not available in memory so it replaces the oldest page slot i.e. 1. —>1 Page Fault. 6 comes, it is also not available in memory so it replaces the oldest page slot i.e. 3 —>1 Page Fault. Finally, when 3 come it is not available so it replaces 0 1- page fault.
- Be lady's anomaly proves that it is possible to have more page faults when increasing the number of page frames while using the First in First Out (FIFO) page replacement algorithm. For example, if we consider reference strings 3, 2, 1, 0, 3, 2, 4, 3, 2, 1, 0, 4, and 3 slots, we get 9 total page faults, but if we increase slots to 4, we get 10-page faults.

ii. Least Recently Used – Consider the page reference string 7, 0, 1, 2, 0,

iii. 3, 0, 4, 2, 3, 0, 3, 2 with 4 page frames. Find number of page faults.

Page reference: 7,0,1,2,0,3,0,4,2,3,0,3,2 No. of page from:4

Total Page Fault = 6

Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots —> 4 Page faults

0 is already there so —> 0 Page fault. when 3 came it will take the place of 7 because it is not used for the longest duration of time in the future.
—>1 Page fault. 0 is already there so —> 0 Page fault. 4 will takes place of 1 —> 1 Page Fault.

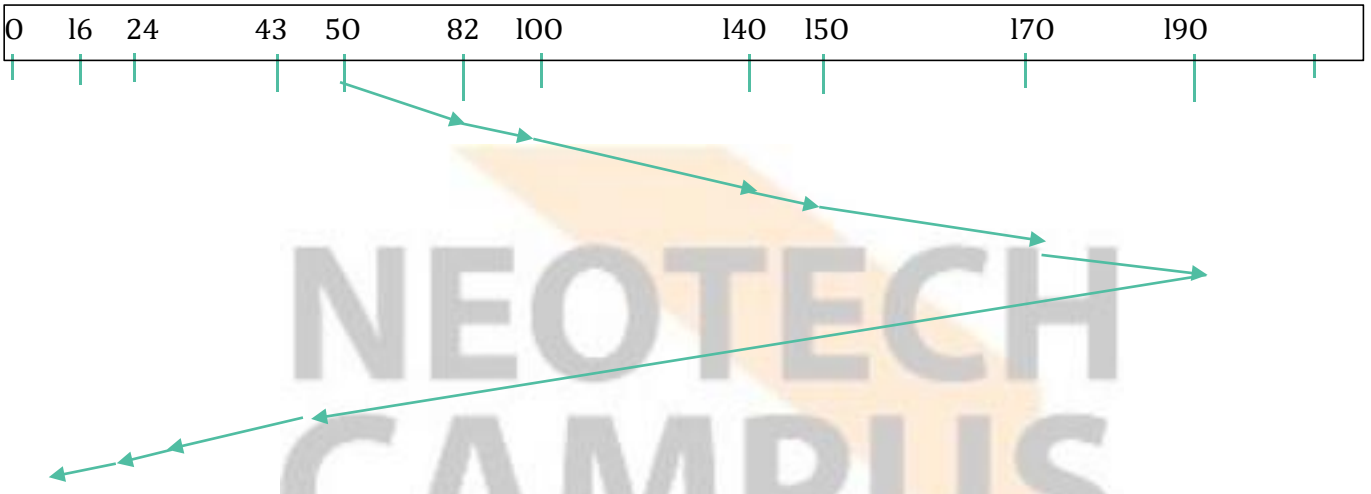
Now for the further page reference string —> 0 Page fault because they are already available in the memory.

Optimal page replacement is perfect, but not possible in practice as the operating system cannot know future requests. The use of Optimal Page replacement is to set up a benchmark so that other replacement algorithms can be analysed against it.

Experiment No: 5

Disk Scheduling Algorithm:

l.Scan: suppose the requests to be addressed are 82,l70,43,l40,24,l6,l90. And the read/write arm is at 50, and it is also given that disk arm should move **"towards the larger value".**

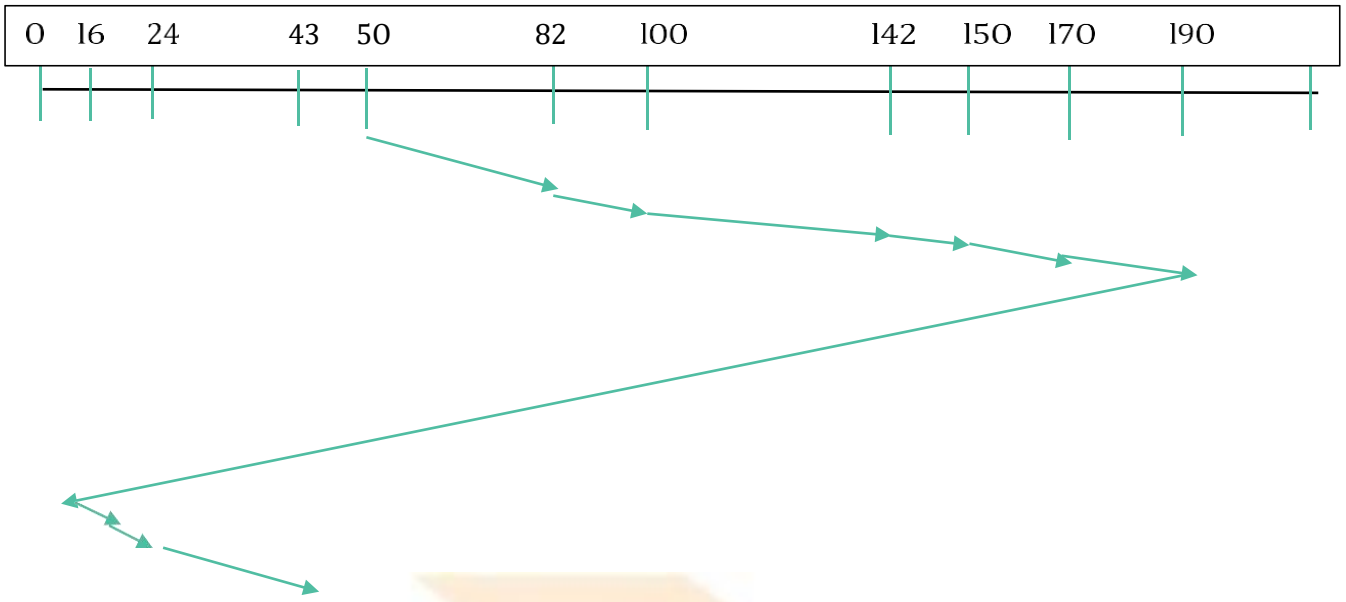


Therefore, the seek time is calculated as:

= (82-50) + (l00-82)+(l40-l00)+(l50-l40)+(l70-l50)+(l90-l70)+(l90-43)+(43-24)+(24-l6)+(l6-0)

= 330

2.CSan: suppose the requests to be addressed are 82,l70,43,l40,24,l6,l90. And the read /write arm is at 50, and it is also given that the disk arm should move **"towards the larger value".**



So, seek time is calculated as:

= (82-50) + (100-82) + (142-100)+(150-142)+(170-150)+(190-170)+(190-0)+(16-0)+(24-16)+(43-24)

= 373

Experiment No: 6

AIM: Test and run basic Unix commands.

➔**Input**

```
yashg@DESKTOP-AJR5RHE MINGW64 ~
$ pwd
```

➔Output

```
c/Users/yashg
```

➔**Input**

```
yashg@DESKTOP-AJR5RHE      MINGW64  ~
$ mkdir yash

yashg@DESKTOP-AJR5RHE      MINGW64  ~
$ cd yash
```

➔Output

```
ashg@DESKTOP-AJR5RHE MINGW64 ~/yash
```

➔**Input**

```
yashg@DESKTOP-AJR5RHE      MINGW64   ~/yash
$ mkdir xyz

yashg@DESKTOP-AJR5RHE      MINGW64   ~/yash
$ dir
```

➔Output

```
$ dir
xyz
```

➔**Input**

```
yashg@DESKTOP-AJR5RHE MINGW64 ~
$ dir
```

➔**Output**

```
123.py
3D\ Objects
4x4.py
ASCII.py
AppData
Application\ Data
Contacts
Cookies
Documents
```

➔**Input**

```
yashg@DESKTOP-AJR5RHE MINGW64 ~
$ who

yashg@DESKTOP-AJR5RHE MINGW64 ~
$ whoami
```

➔**Output**

```
yashg@DESKTOP-AJR5RHE MINGW64 ~
$ whoami
yashg
```

➔**Input**

```
yashg@DESKTOP-AJR5RHE MINGW64 ~
$ date
```

➔**Output**

```
yashg@DESKTOP-AJR5RHE MINGW64 ~
$ date
Sun Jan 22 14:39:41 IST 2023
```

Experiment No: 7

AIM: Test and run advanced Unix commands.

➔**Input**

```
yashg@DESKTOP-AJR5RHE MINGW64
~/OneDrive/Desktop
$ echo "hello world"
```

➔**Output**

```
yashg@DESKTOP-AJR5RHE MINGW64 ~/OneDrive/Desktop
$ echo "hello world"
hello world
```

➔**Input**

```
yashg@DESKTOP-AJR5RHE MINGW64
~/OneDrive/Desktop
$ echo $23+23
```

➔**Output**

```
yashg@DESKTOP-AJR5RHE MINGW64 ~/OneDrive/Desktop
$ echo $23+23
3+23
```

➔**Input**

```
yashg@DESKTOP-AJR5RHE MINGW64
~/OneDrive/Desktop
$ ps
```

➔Output

```
yashg@DESKTOP-AJR5RHE MINGW64 ~/OneDrive/Desktop
$ ps
      PID    PPID    PGID     WINPID   TTY         UID    STIME COMMAND
     1543       1    1543      11648  ?        197609 14:51:41 /usr/bin/mintty
     1544    1543    1544      12668  pty0     197609 14:51:41 /usr/bin/bash
     1676    1544    1676       7620  pty0     197609 23:37:23 /usr/bin/ps
```

➔**Input**

```
yashg@DESKTOP-AJR5RHE MINGW64
~/OneDrive/Desktop
$ ls -a
```

➔Output

```
yashg@DESKTOP-AJR5RHE MINGW64 ~/OneDrive/Desktop
$ ls -a
 ./                                Dev-C++.lnk*                'My Resume.docx'
 ../                              'E-mail ^0 password.docx'    'MySQL 8.0 Command Line Client.lnk'*
'BASIC OPREATING_removed.pdf'      FiveM.lnk*                  'New Microsoft Word Document (3).docx'
'Counter-Strike Global Offensive.url'  'Grand Theft Auto V.url'  'New Microsoft Word Document.docx'
'Dead By Daylight.lnk'*           'Internet Download Manager.lnk'*  'New Text Document (2).txt'
```

➔**Input**

```
yashg@DESKTOP-AJR5RHE MINGW64
~/OneDrive/Desktop
$ cat
```

➔Output

```
yash
yash
goswami
goswami
divyansh
divyansh
himanshu
himanshu
```

➔**Input**

```
yashg@DESKTOP-AJR5RHE MINGW64
~/OneDrive/Desktop
$ ls -i
```

➔**Output**

```
$ ls -i
 9288674231496835 'BASIC OPREATING_removed.pdf'          40532396646348343 'New Microso
 9007199255032744 'Counter-Strike Global Offensive.url'   3659174697709170 'New Text Do
 7318349394753220 'Dead By Daylight.lnk'*                 47006321110705320 'New Text Do
  5348024557614069  Dev-C++.lnk*                          7318349394483397 'New folder'
90071992547436840 'E-mail ^O password.docx'              18295873486218662 'PYTHON LAB
25895697857459377  FiveM.lnk*                            16607023625991925 'PYTHON LAB
13229323905673969 'Grand Theft Auto V.url'               4785074604380441 'PYTHON LAB
 9570149208196985 'Internet Download Manager.lnk'*       9851624185147379 'Rockstar Ga
11821949021912475 'My Resume.docx'                       22517998136932357 'Visual Stud
 4785074604312350 'MySQL 8.0 Command Line Client.lnk'*    2814749767169957  Xbox.lnk*
74309393851625891 'New Microsoft Word Document (3).docx' 32651097298506870  account.sql
```

Experiment No: 8

AIM: Test commands related with File editing with Vi, Vim, gedit, gcc

1. Open a file with vi filename.sh or vim filename.sh.
2. Press i to enter insert mode.
3. Type the program.
4. Press Esc, then type :wq to save & exit.
5. For GUI, use gedit filename.sh.
6. For C programs:
    - Write a program in program.c.
    - Compile: gcc program.c -o program.
    - Run: ./program.

Example Code (program.c):

```
#include <stdio.h>
int main() {
        printf("Hello, World!\n");
        return 0;
}
```

Output: **Hello, World!**

Experiment No: 9

AIM: Create a Shell Script to Read from Command Line and Print "Hello"

Algorithm:

1. Start the script with #!/bin/bash.
2. Read user input using read.
3. Print greeting message using echo.

hello.sh

```
#!/bin/bash
# Script to read input from command line and print Hello
    echo "Enter your name:"
    read name
    echo "Hello $name"
```

Output:

```
Enter your name:
John
Hello John
```

Conclusion: Successfully displayed "Hello" followed by the user's name.

Experiment No: 10

AIM: Create a Shell script to read and display content of a file. And append content of one file to another.

Algorithm:

1. Accept source and destination file names.
2. Display contents of source file using cat.
3. Append source file into destination using >>.
4. Display updated destination file.

Program: fileops.sh

```
#!/bin/bash
# Script to read and display file content, then append to another file

echo "Enter source file name:"
read src

echo "Enter destination file name:"
read dest

echo "Content of $src:"
cat $src
echo "Updated content of $dest:"
cat $dest
```

Output (Example):

```
Enter source file name:
file1.txt
Enter destination file name:
file2.txt
```

Content of file1.txt:
This is file1.

Appending content of file1.txt to
file2.txt... Updated content of file2.txt:

This is file2.
This is file1

Conclusion: Successfully read, displayed, and appended file content.

Experiment No: 11

AIM: Create a Shell script to accept a string in lower case letters from a user, & convert to upper case letters**.**

Algorithm:

1. Accept a string from user.
2. Use tr command to convert lowercase to uppercase.
3. Display result.

Program: upper.sh

```
#!/bin/bash
# Script to convert lowercase string to uppercase

echo "Enter a string in lowercase:"
read str

upper_str=$(echo $str | tr '[:lower:]'
'[:upper:]') echo "Uppercase String:
$upper_str"
```

Output:

```
Enter a string in lowercase:
hello world
Uppercase String: HELLO WORLD
```

Conclusion: Successfully converted lowercase string to uppercase.

Experiment No: 12

AIM: Create a Shell script to add two numbers.

Algorithm:

1. Read two numbers from user.
2. Perform addition using (( )).
3. Display result.

Program: add.sh

```
#!/bin/bash
# Script to add two numbers
echo "Enter first number:" read num1

echo "Enter second number:" read num2
sum=$((num1 + num2)) echo "Sum = $sum"
```

Output:

```
Enter first number:
12
Enter second number:
8
Sum = 20
```

Conclusion: Successfully added two numbers using shell script.